relocated without us having to notify their clients. However, this raises the question of a session. What happens to the information that the old service had about the client? If this information is necessary, it must be kept somehow. We discuss this problem in more detail in the next section.

The fact that a *service can fail* without affecting its caller is an essential characteristic of a distributed, reliable system. This feature is the twin of mobility: while a service is temporarily restarted, location transparency provides users with a consistent facade. Thus, the client can be redirected to another healthy service with the same address, just like any load balancer. Another example you know well by now is an actor's address. The fact that you only use its reference serves the same purpose.

Two other concepts are not directly related to containerization but to the virtues of isolation. The *state* of a service is its most valuable asset and should therefore be passed on with extreme care. This idea was present in the original SOA, and the basic idea was to allow access to a service's data only via the service's API. This way, a user can interact with the service without worrying about how it is handled internally. On the other hand, the data owner can ensure that their data is handled as stated in the specification.

At this point, it's helpful to remember how domain-driven design (DDD) can help you define these boundaries. Keep in mind that it's better if each service has its own database. Isolating each service in its own database prevents the database from being overloaded by another service. The obvious downside is that each database must be managed, which drives up operating costs and often leads to abandoning the idea and sharing databases. A middle ground can be